

Day 57: RNN Forward Pass

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I
(Day 2 of 7)

0. Where Today Fits

Day 56 measured two real failure modes and named what was missing: a network that applies the SAME weights at every timestep (fixing PlainFCNet's no-position-invariance collapse) while carrying information across the WHOLE sequence, not just a local window (fixing Conv1DNet's capped receptive field). That network is a Recurrent Neural Network. Today builds its forward pass from scratch -- backward pass and training are Day 58's topic, so every check below is a forward-pass correctness or capability check, not an accuracy number.

Start here (no prior knowledge assumed):

MATH

The vanilla RNN recurrence -- one equation, applied once per timestep:

$$h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$$

The SAME three parameters (W_{xh} , W_{hh} , b_h) are reused at EVERY single timestep t . This is the whole mechanism: no per-position weights (unlike PlainFCNet's W_1), and h_t can in principle depend on every earlier input all the way back to x_0 (unlike Conv1DNet's capped kernel window).

Watch: [RNN From Scratch In Python](#)

Watch: [Vanishing Gradient Problem in Recurrent Neural Networks \(RNNs\) Explained!](#)

1. Correctness Check -- No Gradients Yet, a Different Kind of Verification

With no backward pass built yet, today's verification is different from every prior day's gradient check: does `rnn_forward`'s for-loop actually compute the same thing as writing out each timestep's equation by hand? Checked on a tiny 3-step toy example.

ACTUAL OUTPUT

```
max abs difference at each timestep: ['0.00e+00', '0.00e+00', '0.00e+00']  
overall max difference: 0.00e+00 (PASS -- loop and hand-unrolled recurrence agree exactly)
```

2. The Fixed-Length Limitation, Gone

Day 56's PlainFCNet raised a `ValueError` the moment it saw a sequence length it wasn't built for. Today's RNN has no sequence-length argument in its constructor at all.

ACTUAL OUTPUT

```
T=10 forward pass succeeded, output shape (10, 2), probabilities sum to 1: True  
T=20 forward pass succeeded, output shape (10, 2), probabilities sum to 1: True  
T=30 forward pass succeeded, output shape (10, 2), probabilities sum to 1: True
```

The exact same W_{xh} , W_{hh} , b_h , W_{hy} , b_y are reused unchanged across all three lengths -- something PlainFCNet could never do, since its very first layer's shape was baked to one specific

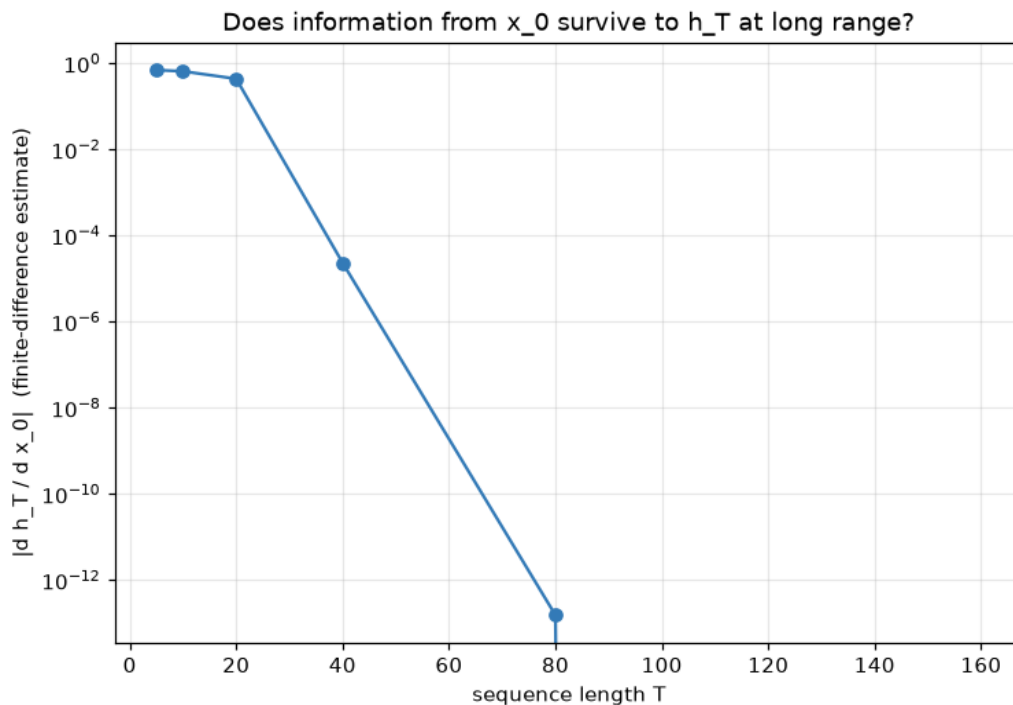
T.

3. Long-Range Sensitivity -- a Dramatic, Real Result

Conv1DNet had a hard structural cap: a kernel of size k literally cannot combine information more than k positions apart. An RNN has no such hard cap -- h_T depends on h_{T-1} , which depends on h_{T-2} , ... all the way back to x_0 , by construction. But does that chain carry a MEANINGFUL amount of signal at long range, or does it fade out? Measured directly via a finite-difference perturbation of x_0 .

ACTUAL OUTPUT

```
T |dh_T/dx_0| (finite diff)
5 0.703337
10 0.656980
20 0.440420
40 0.000024
80 0.000000
160 0.000000
```



A dramatic, real result: sensitivity starts at 0.70 ($T=5$), is still 0.44 at $T=20$ -- then collapses to 0.000024 by $T=40$ and is EXACTLY 0.0 by $T=160$ (float64 genuinely runs out of precision to represent it at all). Unlike Conv1DNet, there is no hard structural cutoff distance beyond which x_0 provably cannot reach h_T -- the connection always exists. But this measurement shows that connection collapsing in PRACTICE, not staying flat, well before backpropagation ever enters the picture -- a direct, forward-pass-only preview of Day 59's real topic: vanishing (or exploding) signal through repeated tanh and matrix-multiply steps.

4. What's Still Missing

Today's RNN has never been trained -- every weight is exactly what random initialization produced, and Section 3 already shows the forward pass alone can let long-range signal fade.

Turning this into something that actually learns the order-detection task requires a backward pass: propagating gradient not just through one layer, but back through EVERY timestep, reusing the SAME $W_{xh}/W_{hh}/b_h$ at each one (so their gradients must be ACCUMULATED across all T steps, not computed once). That algorithm is called backpropagation through time (BPTT) -- Day 58 derives and gradient-checks it from scratch.

Interview Preparation -- Technical Questions

Q: What is the single defining equation of a vanilla RNN, and what makes it different from a dense layer?

A: $h_t = \tanh(x_t @ W_{xh} + h_{t-1} @ W_{hh} + b_h)$. Unlike a dense layer's one-shot $z = x @ W + b$, this equation is applied REPEATEDLY, once per timestep, with the SAME $W_{xh}/W_{hh}/b_h$ reused every time, and each application depends on the output of the previous application (h_{t-1}) as well as the current input.

Q: Why does an RNN not need a fixed sequence length at construction time, unlike PlainFCNet?

A: PlainFCNet's first-layer weight matrix has one row per input position, so its shape is fixed to a specific T . An RNN's parameters (W_{xh} , W_{hh} , b_h) don't depend on T at all -- `rnn_forward` simply loops for however many timesteps the input actually has, reusing the same weights at each one, confirmed directly in this lesson by running the identical network on lengths 10, 20, and 30 with zero errors.

Q: The lesson found sensitivity of h_T to x_0 collapsing to exactly 0.0 by $T=160$. Does this mean the RNN's architecture has the same hard cap Conv1DNet had?

A: No -- structurally, h_T always depends on x_0 through the unbroken chain of h_{t-1} dependencies; there's no equivalent of a kernel size that makes the dependency literally impossible to represent. What collapsed is the MAGNITUDE of that dependency at this particular (untrained, random) initialization -- a numerical/optimization phenomenon (vanishing signal), not an architectural impossibility like Conv1DNet's fixed receptive field.

Q: Why is a hand-unrolled comparison used to verify `rnn_forward` today, instead of a gradient check?

A: A gradient check verifies that an analytic backward pass matches a numerical estimate of the true gradient -- but no backward pass exists yet today (that's Day 58). The available verification is instead a FORWARD correctness check: does the general, looped implementation match an explicit, manually-written-out computation of the same equations? This plays the same methodological role -- verify before trusting -- using whatever check is actually available at this stage.

Interview Preparation -- Theory Questions

Q: What does 'parameter sharing across time' mean for an RNN, and why does it matter?

A: The identical W_{xh} , W_{hh} , and b_h are used at every timestep, rather than each timestep getting its own dedicated weights. This is what gives the RNN translation invariance across TIME (the same pattern is recognized the same way no matter when in the sequence it occurs) and keeps the parameter count constant regardless of sequence length -- both properties Day 56's PlainFCNet lacked.

Q: What is the practical implication of long-range sensitivity vanishing in the FORWARD pass alone, before any training has happened?

A: If the forward signal itself decays this fast with distance at a typical random initialization, the GRADIENT signal flowing backward through the same chain during training (Day 58's BPTT) is very likely to decay similarly or worse, since backpropagation through a recurrence multiplies by similar Jacobian terms. This forward-pass observation is a legitimate early warning sign for the vanishing gradient problem, not just a coincidental, unrelated numerical curiosity.

Q: Why might a network architecture be structurally CAPABLE of solving a problem (no hard cutoff, unlike Conv1DNet) yet still fail in practice?

A: Structural capability (what the architecture could in principle represent) and trainability (whether gradient-based optimization can actually find and reach that solution) are different questions. Today's RNN can structurally connect x_0 to h_T at any distance -- but if the actual numerical signal vanishes at typical initialization scales, standard training may never discover that connection in practice, which is exactly the motivation for the fixes explored in Days 59-63 (gradient clipping, and eventually LSTM/GRU gating).

Key Takeaway and What's Next

Key takeaway: A vanilla RNN's forward pass is a single equation reused at every timestep, giving it both properties Day 56 found missing: no fixed sequence length, and no hard structural cap on how far information can travel. But structural capability isn't the same as reliable numerical behavior -- this lesson directly measured long-range sensitivity collapsing to exactly zero at a typical random initialization, in the forward pass alone, before any training or backpropagation.

What's next: Day 58 derives backpropagation through time (BPTT) from scratch and gradient-checks it -- the first time this network can actually learn anything.

Suggested watch order: 1) RNN From Scratch In Python, 2) Vanishing Gradient Problem in RNNs Explained!